

6. Distributed File System

Q.1 What are desirable Features of good DFS?

⇒

- A DFS is a networked architecture that allows multiple users and applications to access and manage files across various machines as if they were on a local storage device.

- Features of good DFS:-

1) Transparency

⇒ There are four types of transparency:

a) Structure transparency: There is no need for the clients to know about the number or locations of file servers and storage devices.

b) Access transparency: Both local and remote files should be accessible in same manner.

c) Naming transparency: Name of file should remain same when it is moved from one node to other. It's name should be location independent.

d) Replication transparency: If a file is copied on multiple nodes, both the copies of the file and their locations should be hidden from one node to another.

2) User Mobility

⇒ DFS should permit the user to work on different nodes at different times.

- User's home directory should be automatically made available when user logs in on new node.

3) Performance

- ⇒ DFS must give performance same as centralized file system.
- Performance is based on the average amount of time needed to convince the client requests.
- This time covers the CPU time + time taken to access secondary storage + network time access time.

4) Simplicity and ease of use

- ⇒ User interface to file system should be simple and small number of commands should be supported.
- It should support for all types of applications.

5) Scalability

- ⇒ The DFS should also support for growth of nodes and users in the network.
- Such growth should not lead to service loss or performance degradation of the system.
- Service should not be substantially disrupted as the number of nodes and users grows.

6) High Availability

⇒ DFS should continue to function if partial failure occurs in one or more components.

- This failure can be communication link failure, node failure and secondary storage failure.

- Degradation in performance due to failure must be in proportion to that failure.

7) High Reliability

⇒ DFS should support for reliability.

- It should automatically create backup copies of the critical files that can be lost in the failure.

- Users should ensure that there will be no or minimum loss of their data.

8) Data Integrity

⇒ Simultaneous access to ~~mult~~ shared files by many users should guarantee the integrity of data stored in it.

- These access requests to file from many users should be properly synchronized by using proper concurrency control mechanism.

9) Security

⇒ DFS should be secured so that it can offer the privacy to user's data.

- It should implement the security mechanism to protect file data from unauthorized access.

- Secured access right allocation policy should also be supported.

10) Heterogeneity

⇒ In large distributed systems, machines architecture and platform used are different.

- DFS should support for heterogeneity so that users can use different computer platforms for different applications.
- DFS should allow to integrate a new type of node or storage area in simple manner.

Q.2 Explain various File caching schemes.

⇒ File ~~cache~~ caching scheme for DFS should address the decisions like cache location, propagation of modifications and validation of cache.

1) Cache location

⇒ Cache location is the place where the cached data is stored. There can be three possible cache locations -

a) Server's main memory: A cache located in the server's main memory eliminates the disk access cost on a cache hit which increases performance compared to no caching.

- It is easy to implement. Totally transparent to clients.
- The drawback is that, client has to carry out network access for each file access. Hence, network cost is involved.

b) Client disk

⇒ IF cache is located in clients disk it eliminates network access cost but requires disk access cost on cache hit.

- This is slower than having the cache in server's main memory.
- The drawback is that, this policy does not work of diskless workstation.

c) Client's main memory

⇒ A cache located in a client's main memory eliminates both network access and disk access cost.

- This technique is not preferred to a client's disk cache when large caches and increased reliability of cached data are desired.

- It contributes scalability and reliability as access request is served locally on cache-hit.

2) Modification Propagation

- The modification propagation scheme used has a critical effect on the system's performance and reliability.

- Techniques used include:

a) Write-through Policy

⇒ When a cache entry is modified, the new value is immediately sent to the server for updating the master copy of the file.

b) Delayed-write scheme

⇒ To reduce network traffic for writes the delayed-write scheme is used.

- New data value is only written to the cache when an entry is modified and all updated cache entries are sent to the server at a time.

3) Cache Validation Schemes

- The modification propagation policy only specifies when the master copy of a file on the server node is updated upon modification of a cache entry.
- It does not tell anything about when the file data residing in the cache of other nodes is updated.
- It becomes necessary to verify if the data cached at a client node is consistent with the master copy.
- If not, the cached data must be invalidated and the updated version of the data must be fetched again from the server.

There are two approaches:

1) client-initiated approach

⇒ The client starts a validity check in which it contacts the server.

- In this check, client checks consistency of its cached data with the data in master copy.
- Validity checking can be carried out prior to every access or on first access to a file or at regular intervals.
- The validity check is carried out by comparing time of last modification of the cache version of data with server's master copy version.

- If two are same the cached data is considered as up-to-date. Otherwise recent version is accessed from the server.

b) Server-initiated approach

- ⇒ A client informs the file server when opening a file, indicating whether a file is being opened for reading, writing, or both.
- The file server keeps a record of which client has which file open and in what mode.
- The server monitors file usage mode being used by different clients and reacts whenever it detects a potential for inconsistency.
- E.g., if a file is open for reading, other clients may be allowed to open it for reading, but opening it for writing cannot be allowed.
- So also a new client cannot open a file in any mode if the file is open for writing.

Q.3 How are modifications propagated in file caching schemes?

⇒

- When the cache is located on client's nodes, a file's data may simultaneously be cache on multiple nodes.
- It is possible for caches to become inconsistent when the file data is changed by one of the clients and the corresponding data cached at other nodes are not changed or discarded.
- The modification propagation scheme used has a critical effect on file systems performance and reliability.
- Techniques used include -

1) Write-through Policy

⇒ When a cache entry is modified, the new value is immediately sent to the server for updating the master copy of the file.

- This policy is more reliable as little information is lost when client system crashes.
- The write performance is poor as each write access has to wait until the information is sent to the server.
- It is suitable in cases where read to write access ratio is quite large.

2) Delayed write scheme

⇒ To reduce the network traffic for writers the delayed write scheme is used.

- New data value is only written to the cache when a entry is modified and all updated cache entries are sent to the server at a ~~time~~ later time.

- There are three commonly used delayed write approaches:

a) Write on ejection from cache

⇒ Modified data in cache is sent to server only when the cache replacement policy has decided to eject it from client's cache.

- This can result in good performance but there can be a reliability problem since some server data may be outdated for a long time.

b) Periodic write

⇒ This cache is scanned periodically and any cached data that has been modified since the last scan is sent to the server.

c) Write on close

⇒ Modification to cached data is used sent to the server when the client closes the file.

Q.4 Explain synchronization in DFS with its challenges.

⇒

- Synchronization in a DFS refers to maintaining consistency and coordination across multiple replicas or nodes when files are accessed, modified or deleted.
- It ensures that:
 - i) Users see a consistent view of data.
 - ii) Concurrent updates do not conflict
 - iii) Files are properly versioned and updated across the system.

→ Challenges:

1) Concurrency Conflicts

⇒ When multiple clients attempt to read from and write to the same files or data block simultaneously, inconsistencies can arise.

- Implementing robust concurrency control without significantly hurting performance or user experience.

2) Network Latency and Partitioning

⇒ In a distributed environment, messages between nodes may be delayed, dropped or delivered out of order.

- Designing protocols that can handle latency and tolerate partitions.

3) Scalability

⇒ As the number of users, files or nodes increases, maintaining synchronized state across all nodes becomes more resource-intensive.

- Efficient metadata management, distributed locking and load balancing are required to scale synchronization mechanisms effectively.

4) Fault tolerance

⇒ Node crashes or disk failures are inevitable in distributed system.

- Synchronization must account for failure and ensure recovery does not introduce inconsistency.

Implementing recovery protocols that reconcile differences without losing data integrity.

Q.5 Write short note on: NFS

⇒

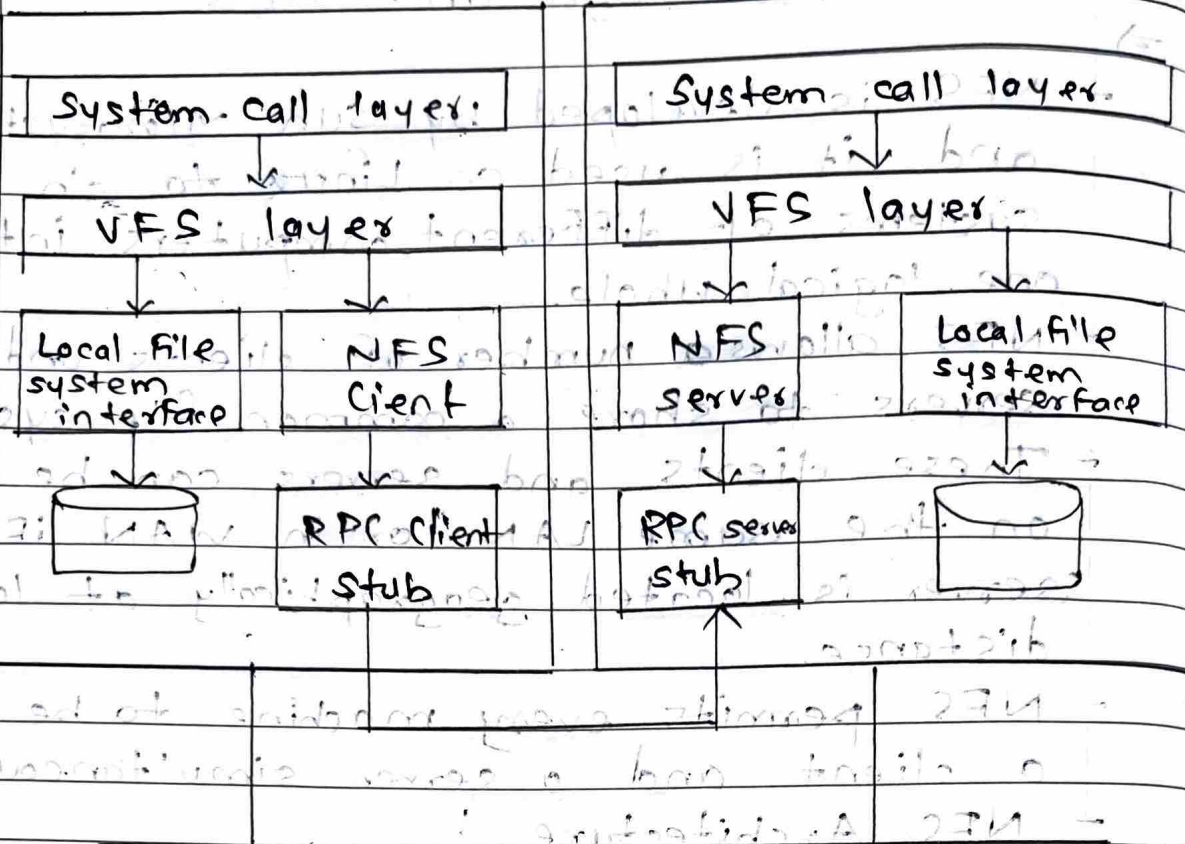
- NFS is developed by Sun Microsystems and it is used on Linux to join file systems of different computers into one logical whole.
- NFS allows a number of clients and servers to share a common file system.
- These clients and servers can be ~~done~~ on the same LAN or in WAN if server is located geographically at long distance.
- NFS permits every machine to be both a client and a server simultaneously.
- NFS Architecture :

Diagram

- NFS uses remote file server model. Remote server manages all the accesses of clients.
- NFS interface offers interface to clients similar to conventional local file system to access the file transparently.
- All the file operations are implemented by servers but their interfaces are offered to the clients.
- NFS also supports upload/download model in which client downloads the file for operations and then uploads it to server so that updated file can be used by other clients.

Client

Server



Network

Client uses system calls of its local operating system. (UNIX).

The local UNIX file system interface is replaced by interface to VFS. (Virtual File System).

The operations related out to VFS interface is either passed out to local file system or to NFS client component.

NFS client then performs RPC to remote server.

VFS hides the differences between different file systems.

NFS server on server side handles incoming requests.

- ~~This means NFS~~
- RPC server stub unmarshals these request and NFS server convert them to regular VFS file operations that are afterward passed to the VFS layer.
- VFS implements local file system where actual files reside.
- In this way, NFS is independent of local file systems, provided, if local file system compliant with file system model of NFS.

• Naming

- NFS naming model offers transparent access for remote file system at server to its requesting clients.
- NFS allows clients to mount part of file system at server.
- The exported directory by server can be maintained in client's local space.
- The drawback of this approach in DFS is that it does not allow sharing files.

- Advantages :

- 1) Easy to use and implement.
- 2) NFS is highly efficient for environments where many clients need access to shared resources.
- 3) Reduces the need for local storage on the each client system.
- 4) NFS can be used on other platforms with proper support.
- 5) Time-saving.

- Disadvantages :

- 1) Security concerns.
- 2) Network dependency.
- 3) Too many clients accessing the server simultaneously can lead to congestion and slower performances.
- 4) No offline access.

Q.6 Discuss Google File System as a scalable DFS.

⇒

- The GFS mainly aims at demanding and rapidly growing needs of Google's search engine and Google web applications.

- Following are the requirements for GFS:

- i) GFS must run reliably on the physical architecture.
- ii) GFS is optimized for patterns of usage within Google, both in terms of types of files stored and the patterns of access to those files.
- iii) GFS must fulfill all the requirements for the Google infrastructure as a whole.

- Following file operations are supported:

1) create - create a new instance of a file.

2) delete - delete an instance of a file.

3) open - open a named file and return a handle.

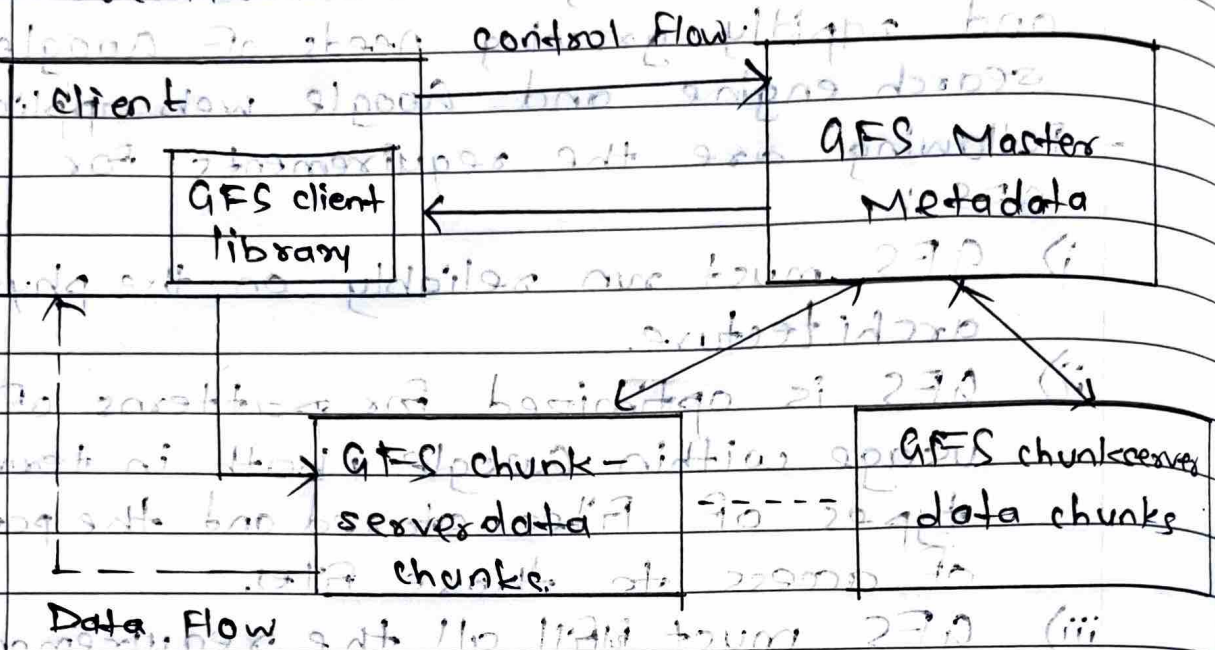
4) close - close a given file specified by a handle.

5) read - read data from a specified file.

6) write - write data to a specified file.

GFS Architecture

- Following figure shows overall GFS architecture.



Data Flow

- In GFS, the storage of files is in fixed-size chunks, where each chunk is 64 megabytes in size.
- This size chosen is very large compared to other file system.
- It offers highly efficient sequential reads and appends of large amount of data.
- GFS provides a mapping from files to chunks and also supports standard operations on files, mapping down to operations on individual chunks.
- Figure shows an instance of GFS file system as it maps onto a given physical cluster.

- GFS cluster has a single master and multiple chunkservers which jointly offer a file service to large numbers of clients concurrently accessing the data.
- The master manages metadata about the file system.
- It defines namespace for files, access control information and mapping of each particular file to the associated set of chunks.
- All chunks are replicated and location of the replicas is maintained in the master.
- Replication provides reliability in case if any software or hardware failure occurs.
- The key metadata is maintained persistently in an operation log.
- This helps in recovery in the event of crashes.

Q.7 Explain naming in distributed systems.

⇒ Naming is the process of identifying and locating resources across the multiple nodes in a distributed system.

Naming is the process of identifying and locating resources across the multiple nodes in a distributed system.

Naming is important because:

1) Resource Identification

⇒ Users and programs need to refer to resources using names rather than physical addresses.

2) Resource Location

⇒ Names must be resolved into addresses so that the system can locate and access the resource.

3) Abstraction

⇒ Names provide an abstraction layer hiding the physical structure or location of the resource.

- In distributed system, there are different types of names and identifiers are used to represent resources.

- Types :-

-1) Human-readable names

-2) Opaque identifier

-3) Hierarchical names

1) Human-readable names

⇒ These names are designed to be easily understood by humans, such as URLs or domain names.

- For e.g.,

2) Opaque identifier

⇒ These identifiers are generally generated by the system and are not meaningful to humans.

- They are used to uniquely identify resources within the system.

3) Hierarchical Names

⇒ These names are structured in a hierarchical manner.

- Each level in the hierarchy represents a different level of organization or categorization.

Q.8 Compare caching and replication

⇒

Caching

Replication

1) Temporarily storing a copy of data close to where it's used.

1) Creating and maintaining multiple full copies of data across system.

2) Often at client-side or intermediate nodes.

2) At multiple server locations.

3) Usually a subset of data.

3) Often entire dataset or large parts of it.

4) Improved read performance and reduce access latency.

4) Improve availability, fault tolerance and sometime performance.

5) Use cases: Web browsers, DNS, lookups.

5) Use Cases: Database replication, CDN.